

Experiment # 07

Implementation of Edge Detection

Objective

1. To find edges in an image

Software

- Python Idle 3.7 or 3.8.5
- PyCharm/VS Code

Theory

In order to find out the boundary of objects in an image or **edge detection**, Sobel filters come in handy.

Sobel Filter

Sobel filter calculates the first order derivative of the image. Horizontal Sobel filter is used to find out the edges along x-axis as it calculates the derivative of an image in x direction while Vertical Sobel filter is used to find out the edges along y-axis. The edges obtained from applying both this filters can then be added to obtain all the edges in an image.

		-1	0	0	-1
		0	1	1	0
	-1	-2	-1	-1	0
	0	0	0	-2	0
	1	2	1	-1	0

Let's walk through an example using Python and the OpenCV library.
Here's the Step-by-Step Example:

1. *Load and Display the Image:* First, we need to load a sample image and display it to understand what we're working with.
2. *Convert to Grayscale:* Convert the image to grayscale as the Sobel operator works on single-channel images.
3. *Apply Gaussian Smoothing (Optional):* Apply a Gaussian blur to reduce noise and make edge detection more robust.

4. *Apply Sobel Operator*: Use the Sobel operator to calculate the gradients in the x and y directions.
5. *Calculate Gradient Magnitude*: Compute the gradient magnitude from the gradients in the x and y directions. A threshold is applied to the gradient magnitude image to classify pixels as edges or non-edges. Pixels with gradient magnitude above the threshold are considered edges.
6. *Normalization*: The gradient magnitude and individual gradients are normalized to the range 0-255 for better visualization.
7. *Display the Resulting Edge Image*: Normalize and display the edge-detected image.

Python Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
image_path = 'flower.jpg' # Replace with your image path
image = cv2.imread(image_path)

# Convert to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Gaussian smoothing (optional)
blurred_image = cv2.GaussianBlur(gray_image, (3, 3), 0)

# Sobel operators
Gx = cv2.Sobel(blurred_image, cv2.CV_64F, 1, 0, ksize=3)
Gy = cv2.Sobel(blurred_image, cv2.CV_64F, 0, 1, ksize=3)

# Gradient magnitude
G = np.sqrt(Gx**2 + Gy**2)

# Normalize to range 0-255
Gx = np.uint8(255 * np.abs(Gx) / np.max(Gx))
Gy = np.uint8(255 * np.abs(Gy) / np.max(Gy))
G = np.uint8(255 * G / np.max(G))

# Display the results
plt.figure(figsize=(15, 10))

# Original image
plt.subplot(2, 2, 1)
```

```
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')

# Gradient in X direction
plt.subplot(2, 2, 2)
plt.imshow(Gx, cmap='gray')
plt.title('Gradient in X direction')
plt.axis('off')

# Gradient in Y direction
plt.subplot(2, 2, 3)
plt.imshow(Gy, cmap='gray')
plt.title('Gradient in Y direction')
plt.axis('off')

# Edge-detected image
plt.subplot(2, 2, 4)
plt.imshow(G, cmap='gray')
plt.title('Sobel Edge Detection')
plt.axis('off')

plt.show()
```

Original Image



Gradient in X direction



Gradient in Y direction



Sobel Edge Detection



- **Laplacian Filter:** Calculates the second derivative of the image to emphasize edges.

- **Unsharp Masking:** A technique where a blurred version of the image is subtracted from the original.

0	1	0	1	1	1
1	-4	1	1	-8	1
0	1	0	1	1	1

The edges can become more pronounced as compared to Sobel filter in some cases.

Python Code

```
# Apply Laplacian Filter
laplacian_filter = cv2.Laplacian(image, cv2.CV_64F)
laplacian_filter = cv2.convertScaleAbs(laplacian_filter) # Convert to 8-bit

# Unsharp Masking
gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=1)
unsharp_image = cv2.addWeighted(image, 1.5, gaussian_blur, -0.5, 0)

# Display Results
plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(laplacian_filter, cmap='gray')
plt.title('Laplacian Filter')
plt.axis('off')

plt.subplot(1, 3, 3)
```

```
plt.imshow(unsharp_image, cmap='gray')
plt.title('Unsharp Masking')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Canny Edge Detection

Canny Algorithm is known for its optimal edge detection capabilities. The algorithm follows a series of steps to reduce noise, detect edges, and improve the accuracy of edge detection.

1. The first step in the Canny edge detection algorithm is to smooth the image using a Gaussian filter. This helps in reducing noise and unwanted details in the image. The Gaussian filter is applied to the image to convolve it with a Gaussian kernel. The Gaussian kernel (or Gaussian function) is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

This step helps to remove high-frequency noise, which can cause spurious edge detection.

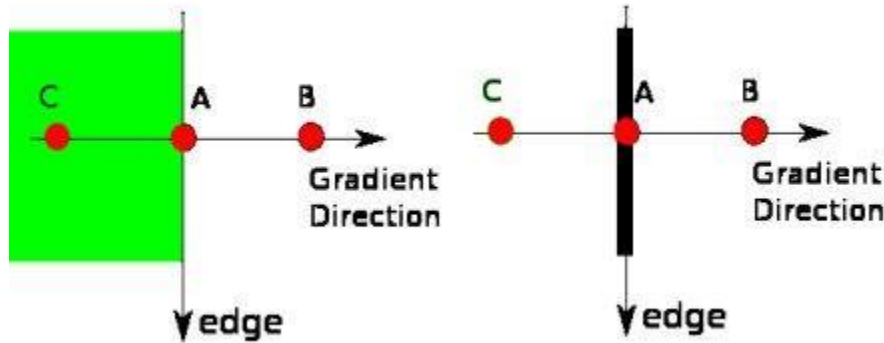
2. *Gradient Calculation:*

After noise reduction, the Sobel operator is used to calculate the gradient intensity and direction of the image. This involves calculating the intensity gradients in the x and y directions (G_x and G_y). The gradient magnitude and direction are then computed using these gradients.

$$\text{Edge_Gradient } (G) = \sqrt{G_x^2 + G_y^2}$$
$$\text{Angle } (\theta) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

3. *Non-Maximum Suppression:* To thin out the edges and get rid of spurious responses to edge detection, non-maximum suppression is applied. This step retains only the local maxima in the gradient direction. The idea is to traverse the gradient image and suppress

any pixel value that is not considered to be an edge, i.e., any pixel that is not a local maximum along the gradient direction.



In the above image, point A is located on the edge in the vertical direction. The gradient direction is perpendicular to the edge. Points B and C lie along the gradient direction. Therefore, Point A is compared with Points B and C to determine if it represents a local maximum. If it does, Point A proceeds to the next stage; otherwise, it is suppressed and set to zero.

4. *Double Thresholding*: After non-maximum suppression, the edge pixels are marked using double thresholding. This step classifies the edges into strong, weak, and non-edges based on two thresholds: high and low. Strong edges are those pixels with gradient values above the high threshold, while weak edges are those with gradient values between the low and high thresholds.

Given the gradient magnitude M and two thresholds T_{high} and T_{low} , the classification can be mathematically expressed as:

Strong Edges:

$$E_{\text{strong}}(i, j) = \begin{cases} 1 & \text{if } M(i, j) \geq T_{\text{high}} \\ 0 & \text{otherwise} \end{cases}$$

Weak Edges:

$$E_{\text{weak}}(i, j) = \begin{cases} 1 & \text{if } T_{\text{low}} \leq M(i, j) < T_{\text{high}} \\ 0 & \text{otherwise} \end{cases}$$

Non-Edges:

$$E_{\text{non-edge}}(i, j) = \begin{cases} 1 & \text{if } M(i, j) < T_{\text{low}} \\ 0 & \text{otherwise} \end{cases}$$

5. *Edge Tracking by Hysteresis*: The final step is edge tracking by hysteresis, which involves traversing the image to determine which weak edges are connected to strong edges. Only the weak edges connected to strong edges are retained, as they are considered true edges. This step ensures that noise and small variations are ignored, resulting in cleaner edge detection.

To simplify the process of Canny Edge detection, OpenCV provides [`cv.Canny\(\)`](#) function.

Following is the code for Canny Edge detection.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
image_path = 'flower.jpg' # Replace with your image path
image = cv2.imread(image_path, cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Gaussian blur to reduce noise
blurred_image = cv2.GaussianBlur(gray_image, (5, 5), 1.4)

# Apply Canny edge detector
edges = cv2.Canny(blurred_image, 100, 200)

# Display the result
plt.figure(figsize=(10, 5))

# Original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')

# Edge-detected image
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Detection')
plt.axis('off')

plt.show()
```

Original Image



Canny Edge Detection



Lab Task:



Apply different edge detection techniques on the above given image and plot result.